

NEUROLEARN

Sistema Operacional de Aprendizagem

Documentação Técnica

Arquitetura, stack, padrões e engenharia

Versão 2.0 · 8 de junho de 2026

NeuroLearn — Documentação Técnica

Versão: 2.1

Última atualização: 2026-06-08

Público: Engenheiros, contribuidores, IA coding agents

Visão Arquitetural

O NeuroLearn é uma aplicação web full-stack com arquitetura server-first (Next.js App Router), backend gerenciado (Supabase) e uma camada de domínio cognitivo pura em TypeScript.

Category	Percentage
Not responsible	35%
Somewhat responsible	45%
Very responsible	15%
Don't know	5%

Stack Tecnológica

Tecnologia	Versão	Papel
Next.js	15.3.3	Framework full-stack (App Router)

Backend / Infraestructura

Tecnologia	Versão	Papel
Supabase	latest	Auth + PostgreSQL + Edge Functions

Testes

Tecnologia	Versão	Papel
Vitest	4.1.8	Testes unitários

Observabilidade

Tecnologia	Versão	Papel
@sentry/nextjs	10.x	Error tracking, session replay, performance tracing

Utilitários

Tecnologia	Papel
docx	Geração de documentos Word

Estrutura de Pastas

```

% % % src/
% % % % app/ # Rotas Next.js (App Router)
% % % % % (app)/ # Grupo de rotas autenticadas
% % % % % % layout.tsx # Verifica sessão + carrega AppContext
% % % % % % % dashboard/
% % % % % % % library/
% % % % % % % focus/
% % % % % % % [contentId]/ # Rota dinâmica para sessão de foco
% % % % % % % review/
% % % % % % % active/
% % % % % % % skills/
% % % % % % % help/
% % % % % % % auth/
% % % % % % % login/page.tsx # Magic Link login
% % % % % % % signup/page.tsx # Cadastro com nome + email
% % % % % % % callback/route.ts # Handler OAuth / Magic Link
% % % % % % % api/
% % % % % % % ai/ # Endpoints de IA
% % % % % % % user/
% % % % % % % % delete/ # DELETE para LGPD
% % % % % % % layout.tsx # Root layout (fontes, tema, LGPD banner)
% % % % % components/
% % % % % % ui/ # Componentes base reutilizáveis
% % % % % % % Button.tsx
% % % % % % % Card.tsx
% % % % % % % Input.tsx
% % % % % % % Textarea.tsx
% % % % % % % FormField.tsx # Wrapper de campo com label + hint + erro
% % % % % % % FormError.tsx # Mensagem de erro com role="alert"
% % % % % % % FormHint.tsx # Texto auxiliar abaixo do campo
% % % % % % % LoadingButton.tsx # Botão com spinner durante submissão
% % % % % % % Toast.tsx # Notificação temporária
% % % % % % % Badge.tsx
% % % % % % % ProgressBar.tsx
% % % % % % % layout/
% % % % % % % Sidebar.tsx # Navegação lateral
% % % % % % % ThemeToggle.tsx # Toggle dark/light
% % % % % % % MigrationBanner.tsx
% % % % % modules/ # Views por feature (lógica de página)
% % % % % % dashboard/DashboardView.tsx
% % % % % % % library/
% % % % % % % LibraryView.tsx
% % % % % % % AddContentModal.tsx
% % % % % % % focus/
% % % % % % % review/ReviewView.tsx
% % % % % % % active/ActiveView.tsx
% % % % % % % skills/SkillsView.tsx
% % % % % % % help/HelpView.tsx
% % % % % % % engine/ # Cognitive Engine (algoritmos puros)
% % % % % % % index.ts # API pública
% % % % % % % spaced-repetition/ # SM-2 + scheduling
% % % % % % % retention/ # Ebbinghaus + forgetting risk
% % % % % % % mastery/ # Mastery score
% % % % % % % skill-evolution/ # Skill progression
% % % % % % % active-learning/ # Pirâmide de Glasser
% % % % % % % cognitive-score/ # Score global
% % % % % % % services/ # Acesso ao Supabase por domínio
% % % % % % % contentsService.ts
% % % % % % % flashcardsService.ts
% % % % % % % skillsService.ts
% % % % % % % sessionsService.ts
% % % % % % % reviewService.ts
% % % % % % % retentionService.ts
% % % % % % % cognitiveEventsService.ts
% % % % % % % localStorageService.ts
% % % % % % % store/
% % % % % % % AppContext.tsx # Estado global + reducer + sync Supabase
% % % % % % % ToastContext.tsx # Toasts (máx 3 simultâneos)
% % % % % % % FocusSessionContext.tsx # Estado global do timer (isRunning)
% % % % % % % components/
% % % % % % % % analytics/
% % % % % % % % PostHogProvider.tsx # Init PostHog + page views + LGPD opt-in/out
% % % % % % % % AnalyticsIdentifier.tsx # Identifica usuário no PostHog e Sentry
% % % % % % % % lgpd/
% % % % % % % % ConsentBanner.tsx # Banner LGPD (nl_lgpd_consent)
% % % % % % % hooks/
% % % % % % % % useAppData.ts
% % % % % % % % useToast.ts
% % % % % % % % useTheme.ts
% % % % % % % % useMigration.ts
% % % % % % % % useAnalytics.ts # Hook type-safe: track(), identifyUser(), resetUser()
% % % % % % % lib/
% % % % % % % % supabase/
% % % % % % % % % client.ts # createBrowserClient
% % % % % % % % % server.ts # createServerClient (cookies)
% % % % % % % % % validation/

```

```

% % % % % schemas.ts # Schemas Zod centralizados
% % % % % security/
% % % % % % % % % rateLimit.ts
% % % % % % % % % validation.ts
% % % % % % % % % sanitize.ts
% % % % % % % % % rbac.ts
% % % % % % % % % logger.ts
% % % % % ai/
% % % % % % % % % client.ts
% % % % % % % % % prompts.ts
% % % % % types/
% % % % % % % % % index.ts # Tipos de domínio
% % % % % % % % % database.types.ts # Gerado pelo Supabase
% % % % % test/
% % % % % % % % % setup.ts # @testing-library/jest-dom
% % % % % tests/
% % % % % % % % % e2e/
% % % % % % % % % global.setup.ts # Auth setup via Supabase Admin API
% % % % % % % % % pages/ # Page Objects
% % % % % % % % % % % % % % % LibraryPage.ts
% % % % % % % % % % % % % % % ReviewPage.ts
% % % % % % % % % % % % % % % utils/helpers.ts
% % % % % % % % % % % % % % % auth.spec.ts
% % % % % % % % % % % % % % % app.spec.ts
% % % % % % % % % % % % % % % ux-01-validation.spec.ts
% % % % % % % % % % % % % % % library.spec.ts
% % % % % % % % % % % % % % % review.spec.ts
% % % % % % % % % % % % % % % accessibility.spec.ts
% % % % % docs/ # Documentação oficial
% % % % % public/
% % % % % % % % % landing.html
% % % % % % % % % % % % % % % middleware.ts # Rate limit + RBAC + session guard
% % % % % % % % % % % % % % % next.config.ts # Security headers + withSentryConfig + CSP expandida
% % % % % % % % % % % % % % % sentry.server.config.ts # Sentry init – server runtime
% % % % % % % % % % % % % % % sentry.edge.config.ts # Sentry init – edge runtime
% % % % % % % % % % % % % % % src/instrumentation.ts # Next.js hook: register() + onRequestError
% % % % % % % % % % % % % % % src/instrumentation-client.ts # Sentry init – browser (Session Replay)
% % % % % % % % % % % % % % % vitest.config.ts
% % % % % % % % % % % % % % % playwright.config.ts
% % % % % % % % % % % % % % % .env.local # Nunca commitar

```

Observabilidade

Sentry v10 — Error Tracking

O Sentry captura automaticamente erros em todos os runtimes:

Runtime	Arquivo de inicialização
Browser	`src/instrumentation-client.ts`

- Session Replay: gravações de sessão com maskAllText e blockAllMedia — 100% em erros, 10% em sessões normais
- onRequestError hook: captura erros em Server Components e Route Handlers
- global-error.tsx: tela de fallback PT-BR para erros de renderização React; reporta ao Sentry via captureException
- Source maps: upload automático no build via SENTRY_AUTH_TOKEN

PostHog — Product Analytics

O PostHog rastreia comportamento de usuário respeitando o consentimento LGPD:

- LGPD: accepted !' opt_in_capturing() | minimal/ausente !' opt_out_capturing()
- Page views: rastreados manualmente via usePathname + useSearchParams (Next.js App Router)
- Identificação: AnalyticsIdentifier chama posthog.identify(userId) ao entrar na área autenticada

Hook `useAnalytics`



Variável	Escopo	Obrigatória
----------	--------	-------------

`NEXT_PUBLIC_SENTRY_DSN`	Público	Para error tracking
--------------------------	---------	---------------------



Todas as vars devem ser adicionadas também no Vercel (Settings !' Environment Variables).

Padrões Arquiteturais

Server Components vs Client Components

- Server Components (padrão no App Router): layouts, páginas de carregamento, leitura de sessão
- Client Components ("use client"): qualquer componente com estado, eventos, hooks, contexto
- Regra: preferir Server Components; usar Client apenas onde necessário

Formulários

Todos os formulários seguem o padrão:



- AppContext: estado global da aplicação (contents, flashcards, skills, user)
 - Reducer puro exportado (`appReducer` + `EMPTY\_STATE`)
 - Side effects assíncronos para sync com Supabase
- ToastContext: fila de toasts (máximo 3 simultâneos, LIFO)
- Sem Redux ou Zustand — React Context é suficiente para o escopo atual

Frontend Architecture

Roteamento



- useTheme() hook expõe theme e toggleTheme()

Componentes UI Base

Todos em src/components/ui/:

Componente	Props principais	Uso
`FormField`	`label, required, hint, children`	Wrapper de campo de formulário

Backend Architecture

Next.js API Routes

Localizadas em src/app/api/:

Rota	Método	Descrição
`/api/user/delete`	DELETE	Exclusão de dados do usuário (LGPD)

Middleware (`middleware.ts`)

Executado em todas as rotas (exceto `_next/static`, imagens, favicon):

- 1. Session guard: rota de app sem sessão !' redirect `/auth/login`
- 2. Auth redirect: rota de auth com sessão !' redirect `/dashboard`
- 3. Rate limiting: 5 req/15min por IP em `/auth/*`
- 4. RBAC: bloqueia `/api/admin/*` para roles insuficientes
- 5. Security logging: registra eventos de segurança em JSON

Clientes Supabase



Banco de Dados

Diagrama de Tabelas


```

users
  id (uuid, FK !' auth.users)
  email, name, role, total_xp, streak, last_study_date

contents
  id, user_id, title, type, author, description
  ContentType: 'book' | 'course' | 'video' | 'article' | 'podcast' | 'other'

flashcards
  id, content_id, user_id
  front, back (texto da pergunta/resposta)
  ef (ease factor), interval, reps, nextReview
  mastery: 'new' | 'learning' | 'review' | 'strong'

skills
  id, name, category, description (habilidades globais)

user_skills
  user_id, skill_id, xp, level (relação usuário !" habilidade)

study_sessions
  id, user_id, content_id
  duration, highlights_count, notes, completed_at

review_cycles
  id, user_id, flashcard_id
  quality (1-5), response_time_ms, retention_before, retention_after
  reviewed_at

retention_snapshots
  id, user_id, flashcard_id
  retention_score, risk_level, snapshot_date

cognitive_events
  id, user_id, event_type, metadata, created_at

```

RLS (Row Level Security)

Todas as tabelas têm RLS ativo. Política padrão:

Tipos de Domínio (`src/types/index.ts`)

Cognitive Engine

O Cognitive Engine é uma biblioteca pura de algoritmos TypeScript em `src/engine/`. Nenhum módulo importa do Supabase ou do Next.js — são funções testáveis em isolamento.

API Pública (`src/engine/index.ts`)

```
// SM-2 aprimorado
export { calculateSM2 } from './spaced-repetition/sm2'
export { buildReviewQueue } from './spaced-repetition/scheduling'

// Retenção
export { calcRetention, calcStability } from './retention/retentionModel'
export { calcRiskLevel } from './retention/forgettingRisk'

// Mastery
export { calcCardMastery, calcContentMastery } from './mastery/masteryScore'

// Skills
export { calcSkillProgression } from './skill-evolution/skillProgression'

// Active Learning (Pirâmide de Glasser)
export { calcActiveLearningScore } from './active-learning/activeLearningScore'

// Score cognitivo global
export { calcCognitiveScore } from './cognitive-score/cognitiveScore'
```

SM-2 Aprimorado

Baseado no algoritmo SuperMemo 2 com bônus por tempo de resposta rápido:

Risco de Esquecimento

1. Filtrar flashcards com nextReview <= agora
2. Ordenar: high != medium != low
3. Desempate: nextReview crescente

Mastery Score (0–100)

Sistema de Revisão Espaçada

Fluxo Completo

```
1. Usuário abre /review
2. ReviewView chama buildReviewQueue(flashcards)
3. Cards due são apresentados um a um
4. Usuário avalia: 1 (blackout) !' 5 (perfeito)
5. calcSM2() recalcula EF, interval, nextReview
6. reviewService.recordReviewCycle() persiste no banco
7. AppContext dispara UPDATE_CONTENT_PROGRESS
8. Fila atualizada no estado local
```

Qualidades de Avaliação (escala 0-5)

Qualidade	Significado
0-2	Falha total / confusão — reinicia interval para 1

Sistema de Autenticação

Fluxo Magic Link



- 5 requisições por IP em 15 minutos para /auth/*
- Retorna HTTP 429 com Retry-After header
- Implementado em memória (edge-compatible)

Segurança

HTTP Security Headers (`next.config.ts`)

Header	Valor
Content-Security-Policy	`default-src 'self'` + exceções necessárias

Sanitização (`src/lib/security/sanitize.ts`)



- Banner de consentimento em src/components/lgpd/ConsentBanner.tsx
- localStorage para persistir preferências
- DELETE /api/user/delete remove dados de todas as tabelas + auth.admin.deleteUser

Estratégia de Testes

Unitários (Vitest)

Configuração (vitest.config.ts):

- Ambiente padrão: node
- Arquivos DOM: // @vitest-environment jsdom no topo do arquivo
- globals: true para matchers do jest-dom
- plugins: [react()] para transpilação JSX
- setupFiles: ['src/test/setup.ts']

Padrão de mock para Supabase:



Projetos.



- SUPABASE_SERVICE_ROLE_KEY (somente Node.js, nunca browser)
- TEST_USER_EMAIL

Comandos de Teste

```
npm run test:unit      # Vitest (260 testes)
npm run test:e2e       # Playwright (multi-project)
npm run test:coverage  # Cobertura com thresholds
```

Thresholds de cobertura:

Métrica	Mínimo
Statements	80%

Estratégia de IA

Configuração

```
export PROMPTS_PATH=src/prompts
export COBERTURA_PATH=coverage
```

Prompts centralizados para:

- Geração de flashcards a partir de conteúdo
- Geração de perguntas contextuais
- Resumo de highlights
- Detecção de lacunas de conhecimento

Fluxo Planejado (Fase 5)

```
npm run build:dev
npm run build:prod
```

Deploy e Variáveis de Ambiente

Variáveis obrigatórias (`.env.local` — nunca commitar)

```
export SUPABASE_URL=
export SUPABASE_ANON_KEY=
export NEXT_PUBLIC_SITE_URL=
export NEXT_PUBLIC_REDIRECT_URL=
```

Configuração Supabase (Dashboard)

- Site URL: `http://localhost:3003` (dev) / URL de produção
- Redirect URLs: `http://localhost:3003/auth/callback`

- Email Auth: habilitado
- Magic Link: habilitado

Build



Convenções de Código

TypeScript

- strict: true em tsconfig.json
- Sem any (use unknown + type guards)
- Interfaces para objetos de dados; type para unions e aliases
- Imports com alias @/ !' src/

Nomenclatura

Tipo	Convenção	Exemplo
Componentes	PascalCase	`FormField.tsx`



Commits (padrão descritivo)



- Apenas onde o "porquê" é não-óbvio
- Sem comentários que explicam "o quê" (o código já faz isso)
- Documentação em português

Performance

Frontend

- Server Components por padrão (zero bundle JS para lógica server)
- Lazy loading de Client Components com dynamic() quando adequado
- CSS Tailwind purged no build (apenas classes usadas)
- Fontes via next/font (sem layout shift)

Banco de Dados

- Queries paralelas no AppContext mount (4 queries simultâneas via Promise.all)
-

- getSession() em vez de getUser() no layout (elimina round-trip de verificação)
- Índices nas colunas user_id e nextReview (colunas de filtro principal)

Cognitive Engine

- Algoritmos síncronos e puros (zero latência de I/O)
- buildReviewQueue roda em memória (sem queries adicionais)

Troubleshooting

Erro 500 "Cannot find module './NNN.js'"

Causa: Build realizado com servidor dev ativo — chunks stale em cache.

Solução:



Magic Link não chega / Rate limit

Causa: Free tier Supabase limita envios de email.

Solução: Aguardar ~1h ou configurar SMTP customizado no dashboard Supabase.

Testes E2E falhando por auth

Causa: tests/e2e/.auth/user.json expirado.

Solução: Deletar o arquivo e rodar npm run test:e2e novamente (global.setup.ts regenera).

`expect is not defined` no Vitest

Causa: globals: true ausente no vitest.config.ts ou setup.ts não importa jest-dom.

Solução: Verificar globals: true e import '@testing-library/jest-dom' em src/test/setup.ts.

IDE mostrando erros TypeScript após edição

Causa: Cache de diagnósticos do IDE (stale).

Solução: Verificar o estado real com npx tsc --noEmit.

Supabase RLS bloqueando query

Causa: Query usando cliente browser mas sem sessão ativa.

Solução: Usar cliente server ou verificar auth.uid() na policy.

Gate de Qualidade

Antes de qualquer entrega:



